



AI-Powered Scientific Assistant for Model Discovery & Simulation

Presented by Mohamed Omari

AI Engineering Bootcamp | Final Project

Ironhack | 29 May 2026

Problem & Motivation

What I built

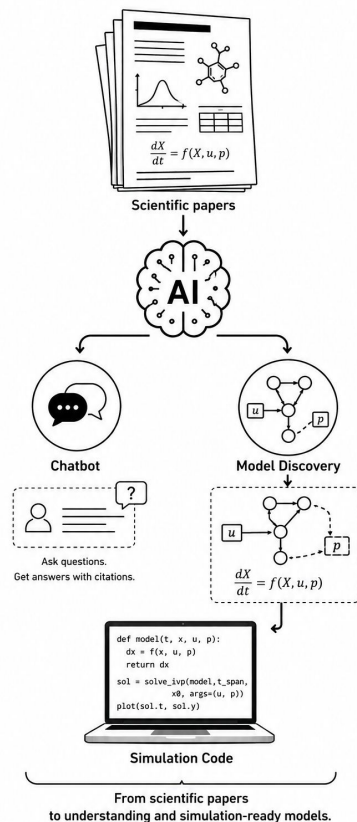
- Scientific paper chatbot for evidence-based Q&A.
- Mathematical model discovery and simulation-ready code.

Why it is needed

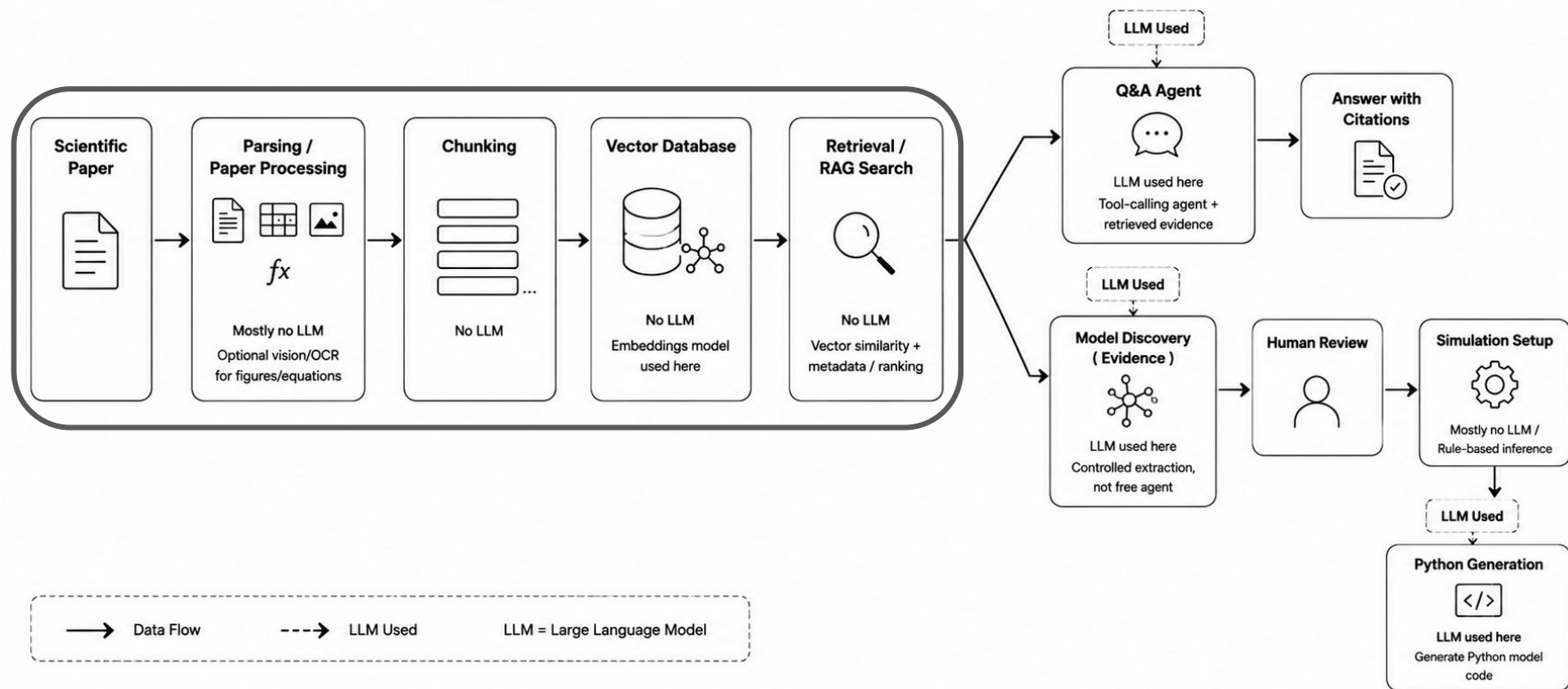
- Model information is scattered across multiple formats.
- Equations, parameters, and mechanisms must be linked correctly.

Why it matters

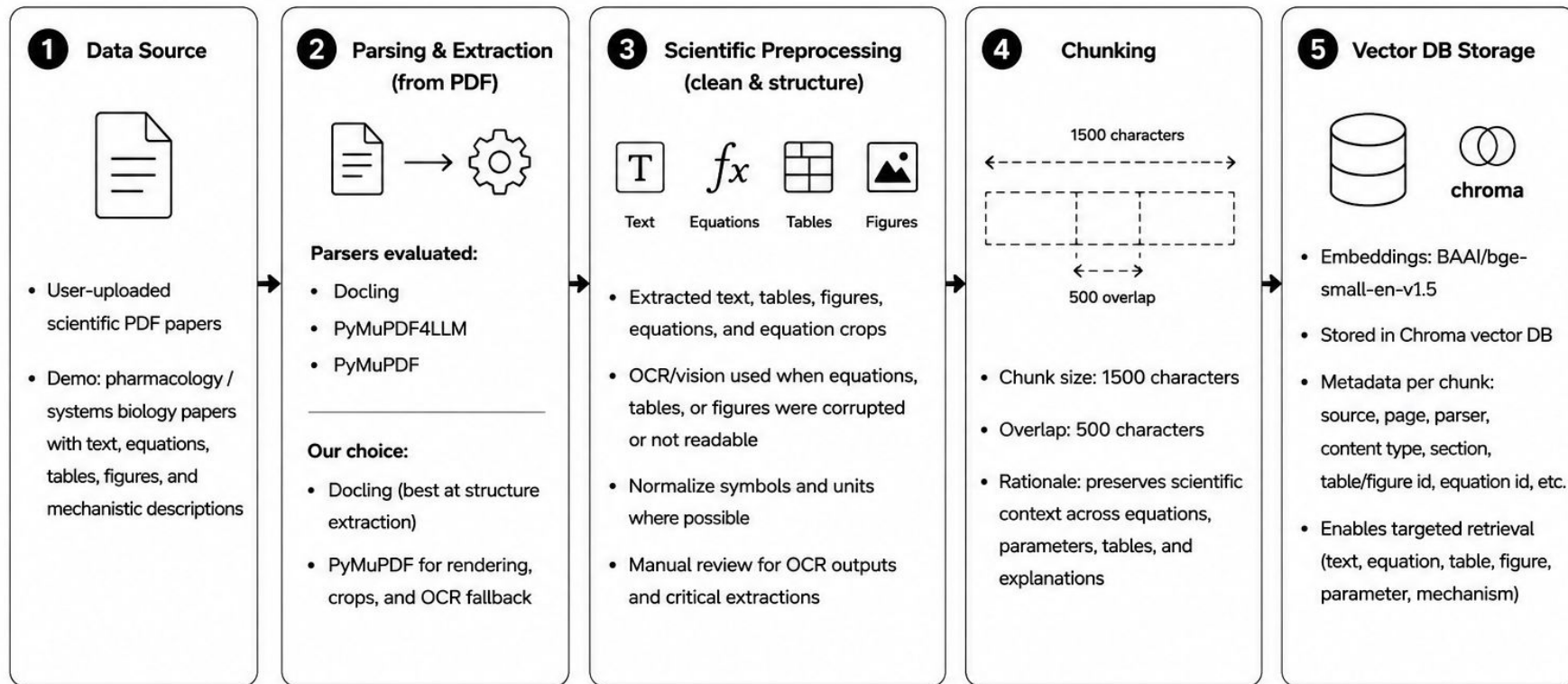
- Goes beyond traditional paper chatbots.
- Bridges scientific Q&A, model validation, and Python simulation.



System Architecture with Streamlit Deployment

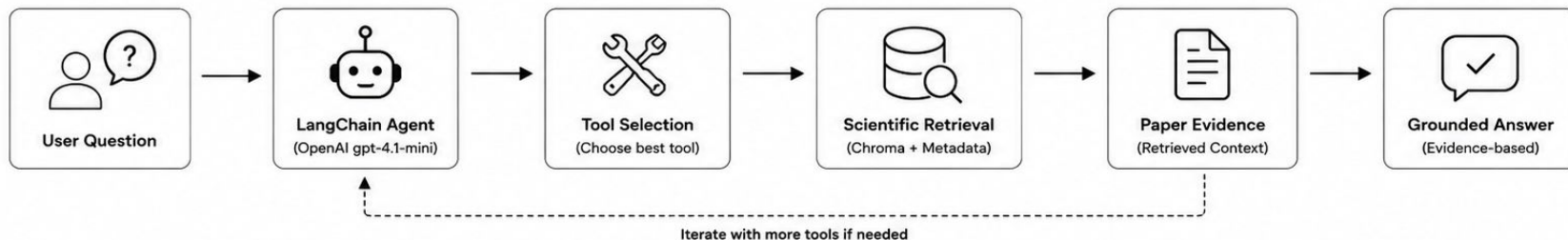


Dataset & Preprocessing



Q&A Agent

1. General Architecture



2. Tooling (Retrieval)

- Text**
aim, abstract, intro, conclusion
- Equations**
equations, ODEs, symbols
- Parameters**
names, values, units
- Mechanisms**
mechanisms, feedback, stimulation
- Tables**
table rows, captions, parameter tables
- Figures**
figure captions and visual evidence
- States / Inputs / Observations**
states, inputs, observations, simulations, assumptions, etc.

All tools use Chroma vector DB, embeddings & metadata filtering

3. Memory

Streamlit Session State (st.session_state)

- ✓ User messages
- ✓ Assistant answers
- ✓ Latest paper answer
- ✓ Active PDF
- ✓ Vector store
- ✓ Current workflow state

UI Memory: stored in session_state

Agent Reasoning Memory: mostly stateless per call

4. Prompting Strategy

Agent System Prompt (rules & guidelines)
tool routing • evidence-only • equation handling
mechanisms • tables • concise synthesis
hallucination prevention

Per-Question Prompt (user question + instructions)
wrapped instruction block
sent to agent for each question

Agent Call
`agent.invoke({"messages": [{"role": "user", "content": user_question}]})`

5. Key Prompt Rules & Answer Style

- ✓ Always retrieve evidence before answering
- ✓ Use only paper context (no inventing)
- ✓ Preserve equation symbols exactly
- ✓ Broad questions → text retrieval first
- ✓ Mechanism/modeling → mechanism + equation (+ parameters if needed)
- ✓ Include relevant equations when central
- ✓ Keep answers concise and non-repetitive
- ✓ Note OCR uncertainty when relevant

Answer Style
Concise, scientific, evidence-gguation-aware.

Typical structure for mechanism questions:

- 1 Mechanism summary
- 2 Relevant equations
- 3 Key variables/parameters
- 4 Evidence / review note

Controlled LLM for Discovery

1 Scientific Paper



Example: PK paper
(mAbs disposition)

2.3. Model Equations

$$\frac{dA_c}{dt} = -\left(\frac{CL}{V_c} + \frac{Q}{V_c}\right)A_c + \frac{Q}{V_p}A_p$$

$$\frac{dA_p}{dt} = \frac{Q}{V_c} A_c - \frac{Q}{V_p} A_p$$

Table 1. Parameter estimates

Parameter	Value	Unit
CL	0.23	L/day
V _c	3.1	L
Q	0.45	L/day
V _p	2.7	L

2 Retrieval Layer



Retrieved evidence
(examples)

- Model equations (Eq. 3, 4)
- Parameter table (Table 1)
- Mechanism description

"A two-compartment model with linear clearance from the central compartment."

3 Controlled LLM (Extraction)



Extracted elements
(structured)

- **Mechanism:** two-compartment PK model with linear clearance from central compartment
- **Equations:** Eq. 3, Eq. 4
- **Parameters:** CL, Vc, Q, Vp
- **Variables:** A_c, A_p
- **Assumptions:** linear PK, well-mixed compartments

4 Structured Output



Example (JSON)

```
{
  "mechanism": "two-compartment
PK model with linear clearance",
  "equations": [
    {
      "eq_id": 3,
      "latex": "dA_c/dt = -(CL/V_c +
Q/V_c) A_c + (Q/V_p) A_p"
    },
    {
      "eq_id": 4,
      "latex": "dA_p/dt = (Q/V_c) A_c
- (Q/V_p) A_p"
    }
  ],
  "parameters": [
    { "name": "CL", "value": 0.23, "unit": "L/"
    { "name": "Vc", "value": 3.1, "unit": "L"
    { "name": "Q", "value": 0.45, "unit": "L/"
    { "name": "Vp", "value": 2.71, "unit": "L"
  ]
},
  "variables": ["A_c", "A_p"],
  "assumptions": ["linear PK",
"well-mixed compartments"]
}
```

5 Human Review



Reviewer checks
and approves

- ✓ Equation symbols correct
- ✓ Parameter units correct
- ✓ Values consistent
- ✓ Mechanism accurate

Status: Approved ✓

6 Simulation Setup



Mapping to model components

- ODEs → model equations
- Parameters → model params
- Variables → model states
- Assumptions → config

Model components

States: A_c, A_p

Parameters: CL, V_G , Q, V_p

Equations: Eq. 3, Eq. 4

7 Python Model Generation



Generated Python code
(example)

```
import numpy as np

def pk_model(t, y, params):
    A_c, A_p = y
    CL, V_c, Q, V_p = params.values()

    dA_c = -(CL/V_c + Q/V_c)*A_c +
            (Q/V_p)*A_p
    dA_p = (Q/V_c)*A_c - (Q/V_p)*A_p

    return [dA_c, dA_p]
```

Evaluation & Iterative Improvement

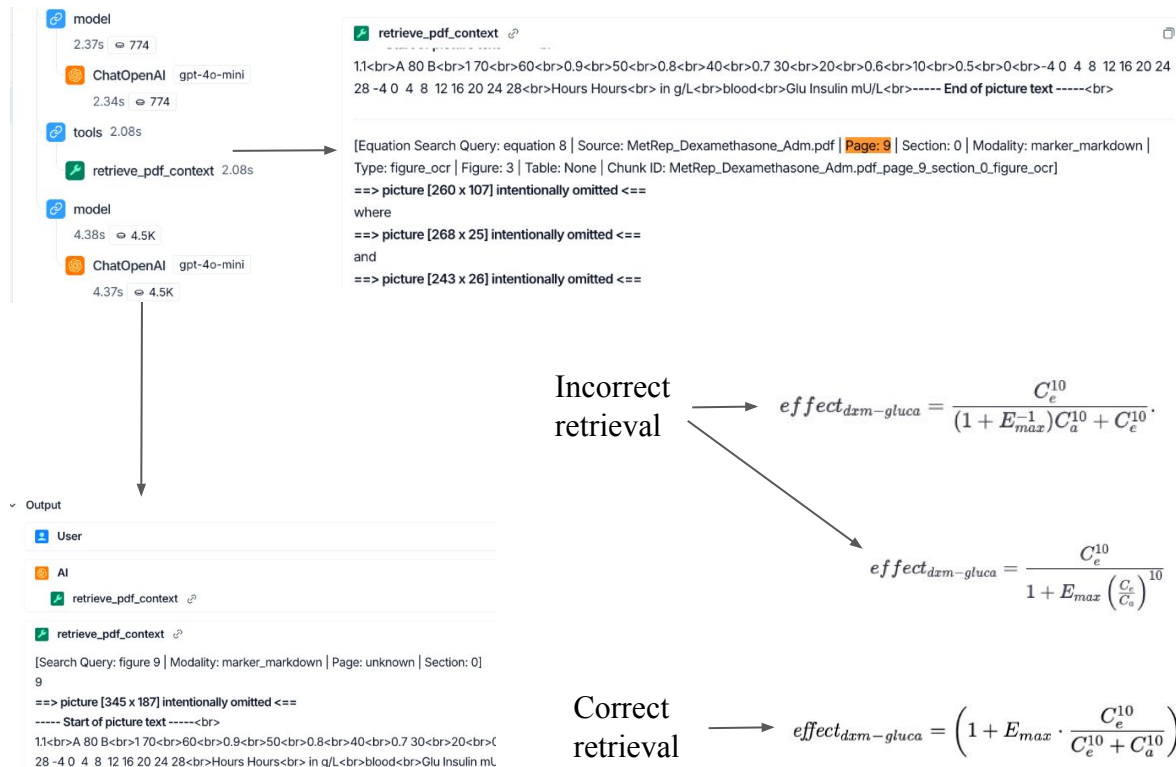
- LangSmith exposed agent tool calls, retrieval steps, and answer generation.

- Scientific PDFs required specialized retrieval for figures, equations, and tables.

- Weak OCR and missing context increased uncertainty.

- To improve, we added specialized retrieval, exact equation matching, OCR/vision fallback, and evidence-grounded prompting.

- Result: more grounded, review-aware scientific answers.



Conclusion

- Lit2Model-AI shows that scientific papers can be transformed into searchable knowledge for Q&A and mathematical model discovery.
- Human validation is important, especially for health-related models, to ensure equations, parameters, and mechanisms are scientifically consistent before simulation.

Ultimate Vision

- Move beyond extraction: from scientific papers to model structures and simulation-ready digital models.
- Build an AI assistant for mathematical modeling that understands e.g. biological interactions, and proposes model structures, and simulates intervention scenarios.



Thank you for the attention!

Tech Stack

 TOOLKITS / LIBRARIES	
	Streamlit App UI
	LangChain / LangGraph Q&A agent and tools
	Chroma Vector database
	HuggingFace Embeddings Local embeddings
	Docling Scientific PDF parsing
	PyMuPDF / fitz PDF text, pages, crops, figures
	PyMuPDF4LLM Alternate PDF-to-markdown parser
	Pydantic Structured schemas
	SymPy Equation parsing support
	NetworkX / PyVis / Mermaid Mechanism graphs and visualizations
	OpenAI API LLM, OCR/vision, STT/TTS
	Gemini API Optional vision/OCR fallback

	 MODELS
	gpt-4.1-mini Q&A agent
	gpt-4o-mini Model discovery, extraction, review helpers, code generation
	gpt-4o High-quality vision OCR option
	gpt-4.1-mini / gpt-4o-mini Cheaper OCR options
	gemini-2.5-flash Optional equation/figure vision
	BAAI/bge-small-en-v1.5 Default embedding model
	text-embedding-3-small Optional OpenAI embedding model
	whisper-1 Speech-to-text (STT)
	tts-1 Text-to-speech (TTS)